DSA Mentor AI - React Login Page Notes

These notes explain the login page in simple English.

1. What Did We Build?

We built a basic login page using React.

The login page has:

- A project heading: DSA Mentor AI
- An email input
- A password input
- A login button
- A small validation message if the user leaves fields empty

This is only the frontend login page.

It does not check real users yet.

Real authentication will come later when we build the backend.

2. How Does The React App Start?

The app starts in this order:

```
index.html -> main.jsx -> App.jsx -> Login.jsx
```

Think of it like this:

- index.html gives React a place to attach the app.
- main.jsx starts React.
- App.jsx loads the main page.
- Login.jsx shows the login form.

3. What Is index.html?

index.html is the main HTML file.

Inside it, we have:

```
<div id="root"></div>
```

React puts our whole app inside this root div.

Interview answer:

> index.html contains the root element where the React application is mounted.

4. What Is main.jsx?

main.jsx is the entry point of the React app.

It connects React with the root div from index.html.

Important code:

```
createRoot(document.getElementById('root')).render(<App />)
```

Meaning:

- Find the HTML element with id root.
- Create a React root there.
- Render the App component inside it.

Interview answer:

> main.jsx is the entry file that renders the root React component into the DOM.

5. What Is A React Component?

A component is a reusable part of the user interface.

In simple words:

```
Component = JavaScript function that returns UI
```

Example:

```
export default function Login() {
  return <h1>DSA Mentor AI</h1>
}
```

Here, Login is a component.

Interview answer:

> A React component is a reusable function or class that returns JSX and represents part of the UI.

6. What Is JSX?

JSX lets us write HTML-like code inside JavaScript.

Example:

```
<h1>DSA Mentor AI</h1>
```

This looks like HTML, but React uses it inside JavaScript.

Interview answer:

> JSX is a syntax extension for JavaScript that allows us to write UI structure inside React components.

7. What Is useState?

useState is used to store values that can change.

In our login page, these values can change:

- Email
- Password
- Error message

Code:

```
const [email, setEmail] = useState(’’)
const [password, setPassword] = useState(’’)
const [error, setError] = useState(’’)
```

Meaning:

- email stores the current email.
- setEmail updates the email.
- password stores the current password.
- setPassword updates the password.
- error stores the current error message.
- setError updates the error message.

Interview answer:

> useState is a React Hook used to store and update state in a functional component. When state changes, React re-renders the component.

8. What Is A Controlled Input?

In our form, React controls the input value.

Example:

```
<input
  value={email}
  onChange={(event) => setEmail(event.target.value)}
/>
```

Meaning:

- value={email} means the input value comes from React state.
- onChange runs when the user types.
- setEmail(event.target.value) saves the typed value in React state.

Interview answer:

> A controlled input is a form input whose value is controlled by React state.

9. What Is An Event?

An event is an action done by the user.

Examples:

- Typing in an input
- Clicking a button
- Submitting a form

In our login page:

```
onChange={(event) => setEmail(event.target.value)}
```

This runs when the user types.

```
onSubmit={handleSubmit}
```

This runs when the form is submitted.

10. Why Do We Use event.preventDefault()?

Normally, when a form is submitted, the browser refreshes the page.

In React, we do not want the page to refresh.

So we write:

```
event.preventDefault()
```

Interview answer:

> preventDefault() stops the default browser form submission so React can handle the form without refreshing the page.

11. How Does Login Submit Work?

Our submit function does four things:

1. Stops page refresh.
2. Clears old error messages.
3. Checks if email or password is empty.
4. Shows an alert if the form is valid.

Code:

```
function handleSubmit(event) {
  event.preventDefault()
  setError('')

  if (!email || !password) {
    setError('Please enter both email and password.')
    return
  }

  console.log('Login submitted:', { email, password })
  alert('Login form works. We will connect it to the backend later.')
}
```

Important point:

This is not real login yet.

For real login, we will send the email and password to the backend.

12. What Is Conditional Rendering?

Conditional rendering means showing something only when a condition is true.

Code:

```
{error && <p className="error-message">{error}</p>}
```

Meaning:

- If error has text, show the error message.
- If error is empty, show nothing.

Interview answer:

> Conditional rendering means displaying UI based on a condition.

13. Why Use htmlFor Instead Of for?

In normal HTML, we write:

```
<label for="email">
```

In React JSX, we write:

```
<label htmlFor="email">
```

Reason:

for is a reserved word in JavaScript, so React uses htmlFor.

14. What Should I Say In An Interview?

You can explain this project like this:

> I built a React login form using functional components and hooks. I used useState to store the email, password, and error message. The form uses controlled inputs, so the input values are managed by React state. I used onChange to update state when the user types, and onSubmit to handle form submission. I also used preventDefault() to stop page refresh and conditional rendering to show validation errors.

15. What Is Done And What Is Not Done?

Done:

- Login UI
- Email input
- Password input
- Basic empty field validation
- React state handling

- Form submit handling

Not done yet:

- Signup page
- Backend
- MongoDB
- Real authentication
- JWT token
- Protected dashboard

16. Next Step

Next, we should build the Signup page.

Signup will use the same concepts:

- Components
- JSX
- useState
- Controlled inputs
- Events
- Form validation
- Conditional rendering

Signup Page Notes

1. What Did We Build?

We added a Signup page.

The Signup page has:

- Name input
- Email input
- Password input
- Confirm password input
- Signup button
- Validation messages

This is still frontend only.

The form does not create a real account yet.

Real account creation will happen later when we connect the backend and MongoDB.

2. Why Did We Add Signup?

In most full-stack apps, users need two authentication screens:

- Login: for existing users
- Signup: for new users

This is why we created a separate Signup.jsx component.

3. What New File Did We Add?

We added:

```
frontend/src/pages/Signup.jsx
```

This file contains the Signup page UI and form logic.

4. Why Is Signup A Separate Component?

We keep Login and Signup separate because each page has different fields and different validation.

This makes the code easier to read and maintain.

Interview answer:

> I separated Login and Signup into different components because each screen has its own UI and logic. This improves readability and maintainability.

5. What Is Happening In App.jsx?

We used state in App.jsx to decide which page to show.

Code idea:

```
const [authPage, setAuthPage] = useState('login')
```

Meaning:

- If authPage is login, show the Login page.
- If authPage is signup, show the Signup page.

This is a simple beginner-friendly way to switch screens before learning React Router.

6. What Are Props?

Props are values passed from one component to another.

Example:

```
<Login onShowSignup={() => setAuthPage('signup')} />
```

Here, onShowSignup is a prop.

It allows the Login component to tell App.jsx to show the Signup page.

Interview answer:

> Props are used to pass data or functions from a parent component to a child component.

7. Parent And Child Components

In our app:

```
App.jsx = parent component
Login.jsx = child component
Signup.jsx = child component
```

App.jsx controls which child component is visible.

This is a common React pattern.

8. Signup Form State

In Signup, we used state for each input:

```
const [name, setName] = useState('')
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
const [confirmPassword, setConfirmPassword] = useState('')
const [error, setError] = useState('')
```

Each input needs its own state because each value can change independently.

9. Signup Validation

We added three validation checks:

1. All fields must be filled.
2. Password must be at least 6 characters.
3. Password and confirm password must match.

This validation happens before sending data to the backend.

Important:

Frontend validation improves user experience, but backend validation is also required for security.

Interview answer:

> I added frontend validation to give quick feedback to the user, but I know backend validation is also necessary because frontend code can be bypassed.

10. What Is Done Now?

Done:

- Login UI
- Signup UI
- Switching between Login and Signup

- Basic form validation
- Reusable auth styling

Not done yet:

- Real signup API
- Real login API
- Password hashing
- JWT authentication
- MongoDB user storage

11. Interview Summary For Signup

You can say:

> I created a Signup component with controlled inputs for name, email, password, and confirm password. I used useState to manage form values and validation errors. I also passed functions as props from the parent App component to switch between Login and Signup screens. This helped me understand component communication, props, and form validation in React.

Dashboard UI Notes

1. What Did We Build After Login?

After Login and Signup, we built the Dashboard page.

The Dashboard is the page a user sees after logging in.

It shows:

- Total solved problems
- Completion percentage
- Current focus topic
- Topic-wise progress
- Recommended problems
- Recent activity

For now, this data is sample frontend data.

Later, this data will come from MongoDB through the backend API.

2. Why Dashboard Comes After Login

In a full-stack app, authentication decides whether a user can access private pages.

Public pages:

- Login
- Signup

Private pages:

- Dashboard
- Problem tracker
- Analytics
- AI assistant

For now, we simulate login by changing state in React.

Later, real login will use JWT tokens from the backend.

3. What New File Did We Add?

We added:

```
frontend/src/pages/Dashboard.jsx
```

This file contains the dashboard UI.

4. What New Concept Did We Learn?

We learned conditional page rendering.

In App.jsx, we used:

```
const [isLoggedIn, setIsLoggedIn] = useState(false)
```

Meaning:

- If isLoggedIn is false, show Login or Signup.
- If isLoggedIn is true, show Dashboard.

Interview answer:

> I used React state to conditionally render the Dashboard after successful login or signup. This simulates protected page behavior before adding real authentication.

5. Why Are We Using Sample Data?

We are using sample data because the backend and database are not ready yet.

This is a common development approach.

First we build the frontend layout with sample data.

Then we replace sample data with real API data later.

Interview answer:

> I first built the Dashboard with static sample data to complete the UI structure. Later, I will replace that data with API responses from the backend.

6. What Is .map() Doing?

In Dashboard, we use .map() to display repeated UI.

Example:

```
topicProgress.map((topic) => {
  return <article key={topic.name}>...</article>
})
```

Meaning:

- Take each topic from the array.
- Create one UI row for that topic.

Interview answer:

> .map() is used in React to render lists by converting each item in an array into JSX.

7. Why Do We Use key In Lists?

React needs a key when rendering lists.

Example:

```
<article key={topic.name}>
```

The key helps React identify which list item changed.

Interview answer:

> A key helps React efficiently update list items by giving each rendered item a stable identity.

8. What Is Done Now?

Done:

- Login page
- Signup page
- Dashboard page
- Frontend-only login simulation
- Logout button
- Basic progress UI
- Recommended problems UI

Not done yet:

- Real authentication
- Backend API
- MongoDB
- Problem tracker CRUD
- Real analytics
- Gemini AI integration

9. Interview Summary For Dashboard

You can say:

> I built a Dashboard component that appears after a successful login or signup. I used React state in App.jsx to simulate authentication and conditionally render the Dashboard. The Dashboard currently uses static sample data for progress, recommendations, and activity. I used .map() to render repeated UI lists and added a logout button to return to the login flow.